

Практическая работа 7–8

ASP.NET Core (.NET 10)

Тема: обработка исключений, HTTP-ошибок и Results API

Сквозное задание: «Учебный сервис HelpDesk.Results»

Паспорт работы	Описание
Формат	Одна интегрированная практическая работа вместо отдельных упражнений по ошибкам и вариантам ответа HTTP.
Цель	Научиться строить Minimal API, которое корректно обрабатывает исключения и HTTP-ошибки, возвращает разные типы ответов через Results API и использует собственный IResult.
Средства	.NET 10 SDK, шаблон Empty Web App (dotnet new web), Visual Studio / Rider / VS Code, браузер, Postman или curl.
Результат	Рабочее веб-приложение, набор тестовых маршрутов и краткий отчет с объяснением порядка middleware, статус-кодов и выбранных Results-методов.

1. Что должен освоить студент

- различать исключения в коде и обычные HTTP-ошибки вроде 404, 401, 403 и 400;
- настраивать обработку исключений в Development и Production без раскрытия технических деталей пользователю;
- подключать UseExceptionHandler(...) и объяснять, почему он должен находиться ближе к началу pipeline;
- подключать UseStatusCodePages(...) или UseStatusCodePagesWithReExecute(...) для понятных ответов на HTTP-ошибки;
- возвращать ответы через Results.Text(...), Results.Content(...), Results.Json(...), Results.Ok(...), Results.BadRequest(...), Results.NotFound(...), Results.Unauthorized(...), Results.StatusCode(...);
- выполнять локальную переадресацию и переадресацию на именованный маршрут через Results API;
- отправлять файл пользователю через Results.File(...) с корректным Content-Type и именем загрузки;
- создавать собственный тип, реализующий IResult, и подключать его через extension method;
- понимать, как статус-код, Content-Type, тело ответа и порядок middleware влияют на результат запроса.

2. Сквозной сценарий задания

Необходимо создать веб-приложение HelpDesk.Results. Это учебный сервис заявок в службу поддержки. Приложение должно показывать список заявок, возвращать данные в разных форматах, обрабатывать ошибочные запросы, имитировать серверное исключение, отдавать файл-инструкцию и демонстрировать разные способы формирования HTTP-ответа через Results API.

Работа должна быть выполнена как Minimal API. Использовать MVC-контроллеры, Razor Pages или готовые CRUD-шаблоны не нужно: цель работы — увидеть поведение middleware и Results API непосредственно в Program.cs и отдельных классах результата/сервиса.

3. Какие темы и где именно отрабатываются

Тема	Что реализует студент	Как проверить
Обработка исключений	Маршрут /throw специально генерирует исключение. В Development можно увидеть страницу разработчика, в Production исключение уходит в общий обработчик.	Запустить приложение в разных средах и обратиться к /throw.
UseExceptionHandler	В Production подключается обработчик исключений, который возвращает понятный ответ 500 без stack trace.	Запрос /throw в Production возвращает контролируемый ответ, а не технические детали.
HTTP-ошибки	Подключается UseStatusCodePagesWithReExecute или собственный UseStatusCodePages-обработчик для 404/401/403.	Запрос /unknown возвращает 404 с понятным телом ответа.
Results.Text / Content	Отдельные маршруты возвращают обычный текст и текст с явно указанным Content-Type/кодировкой.	Проверить /about/text и /about/content.
Results.Json / Ok	Список заявок и найденная заявка возвращаются как JSON/OK-ответ.	Проверить /api/tickets и /api/tickets/1.
BadRequest / NotFound	Ошибочные запросы возвращают корректные статус-коды и	Проверить /api/tickets/999,

/ Unauthorized / StatusCode	понятное тело ответа.	/api/tickets/create без title, /status/unauthorized, /status/custom/418.
Redirects	Старые адреса перенаправляются на новые локальные маршруты или именованные маршруты.	Проверить /redirect/old-tickets и /redirect/ticket/1.
Results.File	Приложение отдает файл-инструкцию из безопасного каталога.	Проверить скачивание /files/readme.
Собственный IResult	Создается HtmlResult и extension method для Results.Extensions.Html(...).	Главная страница или /html/help возвращает HTML через собственный IResult.

4. Функциональные требования к приложению

1. Создать проект на шаблоне Empty Web App под .NET 10 с названием HelpDesk.Results.
2. Создать модель Ticket с полями Id, Title, Status, Priority и CreatedAt.
3. Создать сервис ITicketRepository и реализацию InMemoryTicketRepository, которая хранит минимум 3 учебные заявки.
4. Зарегистрировать сервис заявок в DI до вызова builder.Build().
5. Настроить обработку исключений: для Development использовать developer exception page, для Production — UseExceptionHandler(...).
6. Создать маршрут /throw, который намеренно генерирует исключение для проверки обработчика.
7. Создать обработчик /error/exception, который возвращает ответ 500 без stack trace.
8. Настроить обработку HTTP-ошибок через UseStatusCodePagesWithReExecute("/error/status/{0}") или через делегат UseStatusCodePages(...).
9. Создать /error/status/{code:int}, который возвращает понятное описание HTTP-ошибки.
10. Реализовать обязательные маршруты Results API из таблицы ниже.
11. Сделать минимум один локальный redirect и минимум один redirect на именованный маршрут.
12. Создать файл wwwroot/files/readme.txt и отдавать его через Results.File(...).
13. Создать собственный HtmlResult, реализующий IResult, и extension method для удобного вызова.
14. Сформировать отчет: скриншоты/список запросов, фрагмент Program.cs, объяснение порядка middleware и разницы между исключениями и HTTP-ошибками.

5. Обязательные маршруты и ожидаемое поведение

Запрос	Ожидаемое поведение	Какая тема проверяется
/	Главная HTML-страница с кратким описанием проекта и ссылками на тестовые маршруты. Должна быть сформирована через собственный IResult.	Custom IResult
/about/text	Возвращает простой текст через Results.Text(...).	Text result
/about/content	Возвращает текст через Results.Content(...) с явно заданным MIME-типом и кодировкой.	Content result
/api/tickets	Возвращает список заявок в JSON.	Json / Ok
/api/tickets/1	Возвращает найденную заявку со статусом 200.	Ok + маршрут с параметром
/api/tickets/999	Если заявки нет, возвращает 404 и сообщение.	NotFound
/api/tickets/create?title=Printer&priority=2	Создает учебную заявку и возвращает 201 или 200 с данными созданной заявки.	Created / Ok + Json
/api/tickets/create?priority=2	Без title возвращает 400 с объяснением ошибки.	BadRequest
/status/unauthorized	Возвращает 401.	Unauthorized
/status/forbidden	Возвращает 403 через Results.StatusCode(403) или эквивалентный результат.	StatusCode
/status/custom/418	Возвращает произвольный статус-код 418 через Results.StatusCode(...).	StatusCode
/redirect/old-tickets	Выполняет локальную переадресацию на /api/tickets.	LocalRedirect
/redirect/ticket/1	Выполняет переадресацию на именованный маршрут заявки.	RedirectToRoute
/files/readme	Отдает файл readme.txt с корректным Content-Type и именем загрузки.	File result
/throw	Генерирует исключение для проверки UseExceptionHandler / DeveloperExceptionPage.	Exception handling
/unknown	Возвращает 404 с понятным текстом или JSON, а не пустую	StatusCodePages

6. Минимальный набор собственных классов и структура проекта

Рекомендуемая структура проекта. Названия можно слегка изменить, но смысл должен сохраниться:

```
HelpDesk.Results/
├─ Program.cs
├─ Models/
│   └─ Ticket.cs
├─ Services/
│   ├── ITicketRepository.cs
│   └─ InMemoryTicketRepository.cs
├─ Results/
│   ├── HtmlResult.cs
│   └─ HtmlResultExtensions.cs
├─ wwwroot/
│   ├── files/
│   │   └─ readme.txt
└─ Properties/
    └─ launchSettings.json
```

Допустимые минимальные сигнатуры:

```
public record Ticket(int Id, string Title, string Status, int Priority, DateTime CreatedAt);

public interface ITicketRepository
{
    IEnumerable<Ticket> GetAll();
    Ticket? GetById(int id);
    Ticket Create(string title, int priority);
}

public sealed class HtmlResult : IResult
{
    public HtmlResult(string html) { ... }
    public Task ExecuteAsync(HttpContext context) { ... }
}

public static class HtmlResultExtensions
{
    public static IResult Html(this IResultExtensions extensions, string html) { ... }
}
```

7. Требования к построению pipeline

- Middleware обработки исключений должен быть подключен ближе к началу pipeline, до маршрутов, которые могут выбросить исключение.
- UseStatusCodePagesWithReExecute(...) или UseStatusCodePages(...) должен быть подключен до конечных точек, чтобы обработать 404/401/403, у которых еще нет тела ответа.
- Маршруты /error/exception и /error/status/{code:int} не должны сами генерировать исключения или создавать циклическую переадресацию.
- Если используется UseStatusCodePagesWithRedirects(...), студент должен объяснить, почему клиент получает 302. В основной реализации предпочтительнее ReExecute, потому что исходный статус сохраняется.
- Нельзя отправлять технические детали исключения пользователю в Production: stack trace, имена внутренних классов и абсолютные пути не должны попадать в ответ.
- Нельзя формировать redirect на внешний URL из пользовательского query-параметра без проверки. В этой работе достаточно локальных переадресаций.

- Файл для `Results.File(...)` должен находиться в заранее известном каталоге. Не принимать произвольный путь к файлу из запроса.

8. Минимальные ориентиры по реализации

Фрагменты ниже — не готовое решение, а ориентиры. Студент должен адаптировать их под свой проект и маршруты.

8.1. Обработка исключений и HTTP-ошибок

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<ITicketRepository, InMemoryTicketRepository>();
var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
else
{
    app.UseExceptionHandler("/error/exception");
}

app.UseStatusCodePagesWithReExecute("/error/status/{0}");
```

8.2. Именованный маршрут и RedirectToRoute

```
app.MapGet("/api/tickets/{id:int}", (int id, ITicketRepository repository) =>
{
    var ticket = repository.GetById(id);
    return ticket is null
        ? Results.NotFound(new { message = $"Ticket {id} not found" })
        : Results.Ok(ticket);
}).WithName("ticket-details");

app.MapGet("/redirect/ticket/{id:int}", (int id) =>
    Results.RedirectToRoute("ticket-details", new { id }));
```

8.3. Собственный IActionResult

```
public sealed class HtmlResult : IActionResult
{
    private readonly string _html;
    public HtmlResult(string html) => _html = html;

    public async Task ExecuteAsync(HttpContext context)
    {
        context.Response.ContentType = "text/html; charset=utf-8";
        await context.Response.WriteAsync(_html);
    }
}

public static class HtmlResultExtensions
{
    public static IActionResult Html(this IActionResultExtensions extensions, string html) =>
        new HtmlResult(html);
}
```

9. Отдельные задания по обработке ошибок

1. Проверить `/throw` в Development. В отчете описать, почему такая страница полезна разработчику, но неприемлема для пользователя.
2. Проверить `/throw` в Production или описать, как переключить среду через `launchSettings.json` / переменные окружения. Ответ должен быть 500 без технических деталей.
3. Проверить `/unknown`. Ответ должен содержать статус 404 и понятное сообщение.
4. Проверить `/status/unauthorized` и `/status/forbidden`. Ответы должны иметь статус 401 и 403 соответственно.
5. В отчете объяснить, чем исключение отличается от HTTP-ошибки: исключение возникает внутри кода, а HTTP-ошибка может быть штатным результатом обработки запроса.

10. Отдельные задания по Results API

1. Использовать `Results.Text(...)` и `Results.Content(...)`, причем во втором случае явно задать тип содержимого и кодировку.
2. Использовать `Results.Json(...)` или `Results.Ok(...)` для выдачи списка заявок и одной заявки.
3. Использовать `Results.BadRequest(...)` для некорректного запроса создания заявки.
4. Использовать `Results.NotFound(...)` для несуществующего id.
5. Использовать `Results.Unauthorized(...)` и `Results.StatusCode(...)` минимум на двух отдельных маршрутах.
6. Использовать `Results.LocalRedirect(...)` и `Results.RedirectToRoute(...)`.
7. Использовать `Results.File(...)` для отдачи файла `readme.txt`.
8. Создать свой `HtmlResult` и использовать его хотя бы в одном endpoint.

11. Что должно войти в отчет студента

- скриншот или список успешных запросов ко всем обязательным маршрутам;
- скриншоты `/throw` в Development и Production либо объяснение переключения среды и результат Production-проверки;
- скриншот `/unknown` с 404 и понятным сообщением;
- скриншоты `/api/tickets/999`, `/api/tickets/create` без title, `/status/unauthorized` и `/status/custom/418`;
- фрагмент `Program.cs` с порядком middleware и маршрутов;
- объяснение, почему `UseExceptionHandler` и `UseStatusCodePages` должны находиться до конечных точек;
- краткое объяснение разницы между `Results.Text`, `Results.Content`, `Results.Json` и `Results.File`;
- фрагмент собственного `HtmlResult` и extension method;
- вывод по работе: что было самым важным в понимании статуса ответа, тела ответа и Content-Type.

12. Критерии оценивания

Критерий	Баллы	Что проверяет преподаватель
Проект запускается и содержит обязательные маршруты	10	Работоспособность проекта, корректный набор endpoint и отсутствие критических ошибок запуска.
Обработка исключений настроена корректно	15	Есть <code>/throw</code> , <code>UseExceptionHandler</code> для Production, безопасный ответ 500, понимание Development/Production.
Обработка HTTP-ошибок выполнена корректно	15	<code>UseStatusCodePages</code> или <code>ReExecute</code> , понятный 404, корректные 401/403, отсутствие пустой страницы ошибки.
Results API использован полноценно	20	<code>Text/Content/Json/Ok/BadRequest/NotFound/Unauthorized/StatusCode/Redirect/File</code> применены по смыслу.
Собственный <code>IResult</code> и extension method оформлены корректно	10	<code>HtmlResult</code> реализует <code>ExecuteAsync</code> , задает Content-Type и используется в endpoint.
Модель, сервис и DI выполнены корректно	10	<code>Ticket</code> , репозиторий, минимум 3 записи, регистрация в DI, использование сервиса в маршрутах.
Порядок pipeline и безопасность решений	10	Обработчики стоят до endpoint, нет утечки stack trace, нет небезопасных путей к файлам и открытых redirect.

Отчет и объяснение результата	10	Есть тесты маршрутов, фрагменты кода, объяснение ошибок, статус-кодов, Content-Type и Results API.
-------------------------------	----	--

13. Типовые причины снятия баллов

Ошибка	Как влияет на оценку
Нет UseExceptionHandler или /throw просто падает без контролируемого ответа в Production.	Снимаются баллы за обработку исключений.
Для /unknown возвращается пустая страница или только браузерная стандартная ошибка.	Снимаются баллы за обработку HTTP-ошибок.
Все ответы сделаны обычными строками без Results API.	Сильно снижается оценка за Results API.
Нет собственного IResult или он не используется.	Снимаются баллы за custom IResult.
Файл отдается через произвольный путь из query string.	Снимаются баллы за безопасность и корректность реализации.
Redirect строится на непроверенном внешнем URL из запроса.	Снимаются баллы за безопасность.
Отчет содержит только скриншоты без объяснений.	Снимаются баллы за отчет.
Проект сдан вместе с bin/obj и без исходной структуры.	Может быть снижена оценка за оформление сдачи.

14. Дополнительные задания для сильных студентов

- Сделать /api/tickets/search?status=open и вернуть 204 NoContent, если нет результатов.
- Добавить Results.Problem(...) для серверной ошибки и сравнить его с Results.Json(..., statusCode: 500).
- Сделать HTML-таблицу заявок через собственный HtmlResult.
- Добавить отправку файла как byte[] и сравнить с отправкой файла по пути.
- Добавить логирование исключений через ILogger перед возвратом безопасного ответа пользователю.
- Добавить OpenAPI-метаданные Produces(...) / WithName(...) для основных endpoint.

Итог для студента

Сдать нужно не набор случайных endpoint, а понятную демонстрацию того, как ASP.NET Core формирует HTTP-ответ. В работе должны быть видны статус-код, тело ответа, Content-Type, обработка исключений, обработка штатных HTTP-ошибок и осмысленное применение Results API.